## APPENDIX

The storage price paid by a user is determined by the amount of storage resources consumed and the desired level of fault tolerance. In DELTA, users can flexibly trade off storage cost against reliability by selecting appropriate Reed–Solomon (RS) coding parameters.

Assume that the decentralized storage system consists of $N$ storage nodes, among which $f$ nodes may fail, yielding a node failure rate $p = f/N$. Let $C$ denote the storage price per unit data size per unit time, and let $\tau \in (0, 1)$ denote the target fault tolerance ratio specified by the user. Given a file of size $M$, the total storage cost $Pri$ is given by Eq. (13), and the RS coding parameters $(k, m)$ are determined according to $p$ and $\tau$ as shown in Eq. (14).

$$Pri = \frac{C \cdot M}{1 - \tau} \tag{13}$$

$$k \geq \frac{p \cdot N \cdot (1 - \tau)}{\tau}, \qquad m = \left\lfloor \frac{\tau \cdot k}{1 - \tau} \right\rfloor \tag{14}$$

**Derivation.** Let $n = k + m$ denote the total number of encoded chunks. Given a fault tolerance ratio $\tau$, the system must tolerate up to $\tau n$ faulty chunks. Under RS coding, the original data can be recovered as long as at least $k$ chunks remain available. Therefore, the condition

$$n - \tau n \geq k$$

must hold, which implies

$$m \geq \frac{\tau \cdot k}{1 - \tau}.$$

For a file of size $M$, each chunk has size $M/k$, and the total storage consumption under RS encoding is

$$\frac{M}{k} \cdot (k + m) = \frac{M}{1 - \tau}.$$

Multiplying by the unit storage price $C$, the total storage cost is $Pri = \frac{C \cdot M}{1-\tau}$, as given in Eq. (13). To ensure data recoverability in the presence of node failures, the number of tolerable faulty chunks $\tau n$ must be no less than the number of failed storage nodes $f = pN$, i.e., $\tau \cdot (k + m) \geq f$. Substituting $m \geq \frac{\tau k}{1-\tau}$ yields

$$\frac{\tau \cdot k}{1 - \tau} \geq p \cdot N,$$

which leads to the lower bound on $k$ in Eq. (14).

**Signature verification.** We show the correctness of the signature verification equation (Eq. (8)) by direct derivation. Recall that the signature on chunk $c_\ell$ is defined as

$$\sigma_\ell = \left(\mathtt{H}(t_\ell)^{v_\ell} \cdot g^{\sum_{i=1}^{l} \alpha_i c_{\ell,i}}\right)^x.$$

Then we have

$$\begin{aligned} e(\sigma_\ell, g) &= e\left(\left(\mathtt{H}(t_\ell)^{v_\ell} \cdot g^{\sum_{i=1}^{l} \alpha_i c_{\ell,i}}\right)^x,\, g\right) \\ &= e\left(\mathtt{H}(t_\ell)^{v_\ell} \cdot g^{\sum_{i=1}^{l} \alpha_i c_{\ell,i}},\, g^x\right) \\ &= e\left(\mathtt{H}(t_\ell)^{v_\ell} \cdot \prod_{i=1}^{l} u_i^{c_{\ell,i}},\, y\right), \end{aligned}$$

where $y = g^x$ and $pk = \{u_i = g^{\alpha_i}\}$ constitute the public key. The above derivation shows that the verification equation holds for any correctly generated signature. Therefore, any deviation in the chunk content, metadata, or signature will cause the verification to fail, ensuring the integrity and authenticity of the signed chunk.

**Storage proof verification.** We now show the correctness of the storage proof verification equation (Eq. (10)) based on the definition of the challenge proof. Recall that for a challenged chunk $c_\ell = (c_{\ell,1}, \ldots, c_{\ell,l})$, the storage node computes the data proof $\Lambda_\ell^q = \{\mu_{\ell,i}\}_{i=1}^{l}$, where $\mu_{\ell,i} = s_\ell \cdot c_{\ell,i} \in \mathbb{Z}_p$, and the identity proof $\Theta_\ell^q = \sigma_\ell^{s_\ell}$, with $\sigma_\ell$ being the BLS-style signature on chunk $c_\ell$.

The auditor verifies the proof by checking whether

$$e\big(\mathtt{H}(t_\ell)^{\,v_\ell \cdot s_\ell} \cdot \prod_{i=1}^{l} u_i^{\mu_{\ell,i}},\, y\big) \stackrel{?}{=} e(\Theta_\ell^q, g).$$

For a correctly generated proof, we have

$$\begin{aligned} & e\big(\mathtt{H}(t_\ell)^{\,v_\ell \cdot s_\ell} \cdot \prod_{i=1}^{l} u_i^{\mu_{\ell,i}},\, y\big) \\ =& e\big(\mathtt{H}(t_\ell)^{v_\ell \cdot s_\ell} \cdot \prod_{i=1}^{l} u_i^{s_\ell \cdot c_{\ell,i}},\, y\big) \\ =& e\big((\mathtt{H}(t_\ell)^{v_\ell} \cdot \prod_{i=1}^{l} u_i^{c_{\ell,i}})^{s_\ell},\, y\big) \\ =& e\big((\mathtt{H}(t_\ell)^{v_\ell} \cdot \prod_{i=1}^{l} u_i^{c_{\ell,i}})^{x \cdot s_\ell},\, g\big) \\ =& e(\sigma_\ell^{s_\ell}, g) \\ =& e(\Theta_\ell^q, g). \end{aligned}$$

This confirms that the verification equation holds for any storage node that correctly stores chunk $c_\ell$ together with a valid signature $\sigma_\ell$. Any deviation in the chunk content, metadata, or signature will cause the verification to fail.

**Aggregated proof verification.** We further show the correctness of the aggregated verification equation (Eq. (11)). Let $\mathcal{I}$ denote the set of challenged chunk indices and $\{\beta_\ell\}_{\ell \in \mathcal{I}} \subset \mathbb{Z}_p$ be random aggregation coefficients chosen by the auditor. The aggregated data proof is defined as

$$\mu_i = \sum_{\ell \in \mathcal{I}} \beta_\ell \cdot \mu_{\ell,i} = \sum_{\ell \in \mathcal{I}} \beta_\ell \cdot s_\ell \cdot c_{\ell,i},$$

and the aggregated identity proof is

$$\Theta^q = \prod_{\ell \in \mathcal{I}} \Theta_\ell^{q\,\beta_\ell} = \prod_{\ell \in \mathcal{I}} \sigma_\ell^{s_\ell \cdot \beta_\ell}.$$

The auditor verifies whether

$$e\big(\prod_{\ell \in \mathcal{I}} \mathtt{H}(t_\ell)^{\,v_\ell \cdot s_\ell \cdot \beta_\ell} \cdot \prod_{i=1}^{l} u_i^{\mu_i},\, y\big) \stackrel{?}{=} e(\Theta^q, g).$$

For correctly generated proofs, we have

$$
\begin{aligned}
& e\Big(\prod_{\ell\in\mathcal{I}} \mathtt{H}(t_\ell)^{\,v_\ell\cdot s_\ell\cdot\beta_\ell}\cdot\prod_{i=1}^{l} u_i^{\mu_i},\, y\Big)\\
&= e\left(\prod_{\ell\in\mathcal{I}} \mathtt{H}(t_\ell)^{v_\ell\cdot s_\ell\cdot\beta_\ell}\cdot\prod_{i=1}^{l} u_i^{\sum_{\ell\in\mathcal{I}}\beta_\ell\cdot s_\ell\cdot c_{\ell,i}},\, y\right)\\
&= e\left(\prod_{\ell\in\mathcal{I}} (\mathtt{H}(t_\ell)^{v_\ell}\cdot\prod_{i=1}^{l} u_i^{c_{\ell,i}})^{x\cdot s_\ell\cdot\beta_\ell},\, g\right)\\
&= e\left(\prod_{\ell\in\mathcal{I}} \sigma_\ell^{s_\ell\cdot\beta_\ell},\, g\right)\\
&= e(\Theta^q, g).
\end{aligned}
$$

Thus, the aggregated verification equation holds, allowing the auditor to verify multiple storage proofs using a single pairing check.

**Signature increment.** We now show the correctness of the incremental signature update. Recall that $\Delta c_\ell = (\Delta c_{\ell,1},\ldots,\Delta c_{\ell,l})$ is the chunk update increment, and $c'_\ell = c_\ell + \Delta c_\ell$ is the updated chunk. The corresponding metadata changes from $\langle v_\ell, t_\ell\rangle$ to $\langle v'_\ell, t'_\ell\rangle$. The updated signature $\sigma'_\ell$ is obtained by multiplying the original signature $\sigma_\ell$ with the signature increment $\Delta\sigma_\ell$, as shown below:

$$
\begin{aligned}
\sigma_\ell\cdot\Delta\sigma_\ell &= (\mathtt{H}(t_\ell)^{v_\ell}\cdot g^{\sum_{i=1}^{l}\alpha_i\cdot c_{\ell,i}})^x\cdot\left(\frac{\mathtt{H}(t'_\ell)^{v'_\ell}}{\mathtt{H}(t_\ell)^{v_\ell}}\cdot g^{\sum_{i=1}^{l}\alpha_i\cdot\Delta c_{\ell,i}}\right)^x\\
&= \left(\mathtt{H}(t'_\ell)^{v'_\ell}\cdot g^{\sum_{i=1}^{l}\alpha_i\cdot(c_{\ell,i}+\Delta c_{\ell,i})}\right)^x\\
&= \left(\mathtt{H}(t'_\ell)^{v'_\ell}\cdot g^{\sum_{i=1}^{l}\alpha_i\cdot c'_{\ell,i}}\right)^x\\
&= \sigma'_\ell.
\end{aligned}
$$

This shows that the incrementally updated signature $\sigma'_\ell$ is algebraically identical to a signature generated by directly signing the updated chunk $c'_\ell$ with the new metadata, thereby ensuring the correctness of the incremental update mechanism.

We establish the security and correctness of DELTA from two perspectives: the auditing scheme and the protocol-level design.

**Auditing scheme.** We first analyze the security of the auditing scheme. Specifically, we prove Proposition 1, as introduced in Section V-B, by showing that the proposed construction is resistant to signature forgery, proof forgery, generation attacks, and replay attacks under standard cryptographic assumptions.

*Proposition 1:* The signature scheme, the signature increment mechanism, and the storage proof protocol described in Section V-A are unforgeable.

We prove this proposition through a sequence of theorems, each addressing a specific class of attacks.

*Theorem 1 (Unforgeability of the signature):* For any adversary, it is computationally infeasible to forge the signature $\sigma = (\mathtt{H}(t)^v\cdot g^{\sum_{i=1}^{l}\alpha_i\cdot c_i})^x \in \mathbb{G}_1$ or $\sigma = (\mathtt{H}(t)^v\cdot\prod_{i=1}^{l}u_i^{c_i})^x \in \mathbb{G}_1$ with $x$ unknown, if the group $\mathbb{G}_1$ is chosen such that the discrete logarithm problem is hard.

If the attacker can forge the signature, he must be able to compute $x$. However, if $\mathbb{G}_1$ is chosen as a group for which the discrete logarithm problem is hard, this means that even given $g$ and $g^x$, computing $x$ is computationally infeasible. Therefore, the attacker cannot forge the signature.

*Theorem 2 (Unforgeability of the signature increment):* For any adversary, it is computationally infeasible to forge the signature increment $\Delta\sigma = \left(\frac{\mathtt{H}(t')^{v'}}{\mathtt{H}(t)^{v}}\cdot g^{\sum_i^l\alpha_i\cdot\Delta c_i}\right)^x$ with $x$ unknown, if the group $\mathbb{G}_1$ is chosen such that the discrete logarithm problem is hard.

The proof is identical to the proof of Theorem 1.

*Theorem 3 (Unforgeability of the storage proof):* In the storage auditing scheme, it is computationally infeasible for the SN to forge a storage proof to pass the verification.

The Auditor sends a challenge $chal_\ell = \langle fid, \ell, q, s_\ell\rangle$ to the SN. To pass the verification with Eq. (10), the SN should generate a storage proof $\langle\Lambda_\ell, \Theta_\ell\rangle$ on a correct encoded chunk $c_\ell = \{c_{\ell,i}|1\le i\le l\}$, where $l$ is the length of $c_\ell$ and $c_{\ell,i}$ is the $i$-th element in $c_\ell$. Assuming the SN can generate a proof $\langle\Lambda'_\ell, \Theta_\ell\rangle$ that passes the verification using an incorrect encoded chunk $c'_\ell = \{c'_{\ell,i}|1\le i\le l\}$. Define $\Delta c_\ell = c'_\ell - c_\ell = \{\Delta c_{\ell,i}|\Delta c_{\ell,i} = c'_{\ell,i} - c_{\ell,i}, 1\le i\le l\}$, and at least one element of $\Delta c_\ell$ is non-zero. If the proof $\langle\Lambda'_\ell, \Theta_\ell\rangle$ can pass the verification performed by the Auditor, then according to Eq. (9) and Eq. (10), we have

$$
\begin{aligned}
& e(\Theta_\ell,\, g)\\
=& e\left(\mathtt{H}(t_\ell)^{\,v_\ell\cdot s_\ell}\cdot\prod_{i=1}^{l}u_i^{s_\ell\cdot c'_{\ell,i}},\, y\right)\\
=& e\left(\mathtt{H}(t_\ell)^{\,v_\ell\cdot s_\ell}\cdot g^{\sum_{i=1}^{l}\alpha_i\cdot c'_{\ell,i}\cdot s_\ell},\, y\right).
\end{aligned}
\tag{15}
$$

Moreover, $\langle\Lambda_\ell, \Theta_\ell\rangle$ is the correct proof, so we have

$$
\begin{aligned}
e(\Theta_\ell,\, g) &= e\left(h(t_\ell)^{\,v_\ell\cdot s_\ell}\cdot\prod_{i=1}^{l}u_i^{s_\ell\cdot c_{\ell,i}},\, y\right)\\
&= e\Big(h(t_\ell)^{\,v_\ell\cdot s_\ell}\cdot g^{\sum_{i=1}^{l}\alpha_i\cdot c_{\ell,i}\cdot s_\ell},\, y\Big)
\end{aligned}
\tag{16}
$$

According to Eqs. (15)–(16) and the properties of bilinear maps that $e(g_1, g_2)^a = e(g_1^a, g_2)$, we have

$$
g^{\sum_{i=1}^{l}\alpha_i\cdot c'_{\ell,i}} = g^{\sum_{i=1}^{l}\alpha_i\cdot c_{\ell,i}} \tag{17}
$$

Substitute $c'_{\ell,i} = \Delta c_{\ell,i} + c_{\ell,i}$ into Eq. (18), we have

$$
g^{\sum_{i=1}^{l}\alpha_i\cdot\Delta c_{\ell,i}+\alpha_i\cdot c_{\ell,i}} = g^{\sum_{i=1}^{l}\alpha_i\cdot c_{\ell,i}} \Rightarrow g^{\sum_{i=1}^{l}\alpha_i\cdot\Delta c_{\ell,i}} = 1 \tag{18}
$$

Given that $\forall\alpha_i\in sk, \alpha_i\neq 0$, then we get $\forall\Delta c_{\ell,i}\in\Delta c_\ell, \Delta c_{\ell,i} = 0$, which contradicts the assumption that at least one element of $\Delta c_\ell$ is non-zero. This proves the unforgeability of the storage proof.

*Theorem 4 (Immunity from generation attacks):* In the storage auditing scheme, the SN cannot generate a storage proof to pass the verification without the specified encoded blocks.

The proof of this theorem are the same as that of Theorem 3. The SN cannot generate a verifiable storage proof by forging any checked encoded chunk $c_\ell$.

*Theorem 5 (Immunity from replay attacks):* In the storage auditing scheme, SN cannot generate a storage proof to pass the verification using the former encoded chunks and metadata.

The Auditor sends a challenge $chal_\ell = \langle fid, \ell, q, s_\ell \rangle$ to the SN. Assume that the SN generates a storage proof $\pi'_\ell = (\Lambda'_\ell, \Theta'_\ell)$ using an outdated version $o$ ($o < q$) of the encoded chunk $c_\ell$, i.e.,

$$\begin{aligned} &\Lambda'_\ell = \{\mu'_i \in \mathbb{Z}_p \mid \mu'_i = s_\ell \cdot c^o_{\ell,i},\ 1 \le i \le l\}, \\ &\Theta'_\ell = \sigma_\ell^{o\ s_\ell}. \end{aligned} \tag{19}$$

After receiving $\pi'_\ell$, the Auditor first computes the right side of Eq. (10). According to the properties of bilinear maps, it can be rewritten as follows.

$$\begin{aligned} &e\big(\mathtt{H}(t^q_\ell)^{v^q_\ell \cdot s_\ell} \cdot \prod_{i=1}^{l} u_i^{\mu'_{\ell,i}},\ y\big) \\ =&e\big(\mathtt{H}(t^q_\ell)^{v^q_\ell \cdot s_\ell} \cdot \prod_{i=1}^{l} u_i^{c^o_{\ell,i} \cdot s_\ell},\ y\big) \\ =&e\big(\mathtt{H}(t^q_\ell)^{v^q_\ell \cdot s_\ell} \cdot g^{\sum_{i=1}^{l} \alpha_i \cdot c^o_{\ell,i} \cdot s_\ell},\ y\big) \end{aligned}$$

Further, the Auditor computes the left side of Eq. (10). According to the properties of bilinear maps, it can be rewritten as follows.

$$\begin{aligned} &e\big(\Theta'_\ell, g\big) \\ =&e\big(\sigma_\ell^{o\ s_\ell}, g\big) \\ =&e\big(\big(\mathtt{H}(t^o_\ell)^{v^o_\ell} \cdot g^{\sum_{i=1}^{l} \alpha_i \cdot c^o_{\ell,i}}\big)^{x \cdot s_\ell}, g\big) \\ =&e\big(\mathtt{H}(t^o_\ell)^{v^o_\ell \cdot s_\ell} \cdot g^{\sum_{i=1}^{l} \alpha_i \cdot c^o_{\ell,i} \cdot s_\ell}, g\big) \end{aligned}$$

If the verification is assumed to be passed, then $\mathtt{H}(t^o_\ell)^{v^o_\ell} = \mathtt{H}(t^q_\ell)^{v^q_\ell}$, i.e., $v^o_\ell = v^q_\ell \wedge t^o_\ell = t^q_\ell$. According to the assumption that $o < q$, we have $v^o_\ell \neq v^q_\ell \ \wedge \ t^o_\ell \neq t^q_\ell$, which contradicts the derived result. This completes the proof of the theorem.

**Protocol.** As discussed in Section V-B, the correctness of the protocol hinges on two key properties: (i) every read operation should always return the most recent committed file version, and (ii) storage auditing should be performed on a consistent and latest committed chunk snapshot. In the following, we formally prove that DELTA satisfies these properties under concurrent updates and asynchronous auditing.

*Proposition 2:* At any time, every read operation retrieves the most recent committed file version $F^q$.

A file encoded with RS coding is recoverable as long as at least $k$ valid encoded chunks from the same chunk snapshot are available. Hence, to guarantee correct read operations, it suffices to ensure that at any time at least $k$ valid chunks belonging to the latest committed snapshot $C^q$ are preserved across storage nodes.

This condition is ensured jointly by the version consistency control and the auditing mechanism in DELTA. On the one hand, the two-slot version buffering mechanism guarantees that once a snapshot $C^q$ is committed, its chunks stored in the committed slot are never overwritten by concurrent updates. Update increments for newer versions are buffered separately and merged only after the auditor confirms the update, thereby preserving the integrity of the committed snapshot throughout the update process.

On the other hand, continuous storage auditing enforces that storage nodes correctly store valid chunks belonging to the latest committed snapshot $C^q$. Nodes that fail to provide valid storage proofs are detected and penalized, ensuring that at least $k$ honest storage nodes retain valid chunks of $C^q$ at all times. Combining these guarantees, the system always maintains sufficient valid chunks of the latest committed snapshot, and every read operation can therefore successfully reconstruct the most recent committed file version $F^q$.

*Proposition 3:* Storage auditing in DELTA is always performed on a consistent and latest committed chunk snapshot $C^q$.

DELTA adopts an asynchronous auditing protocol that allows auditing to proceed concurrently with data updates without blocking normal system operations. Under this protocol, the auditor always targets a committed chunk snapshot $C^q$ when issuing audit challenges. To handle scenarios where updates progress rapidly, DELTA further incorporates a piggybacked auditing mechanism. If an audit for snapshot $C^q$ is still pending when a new update is initiated, the auditor attaches the pending audit challenge to the `PREPARE` message of the update protocol. This design guarantees that storage nodes generate the required storage proofs for $C^q$ before the corresponding chunk state could be overwritten by subsequent updates.

Consequently, auditing of snapshot $C^q$ always completes while the necessary chunk data remains locally available at storage nodes, ensuring that all proofs are generated with respect to a mutually consistent and latest committed snapshot. Finally, DELTA embeds the version number $v_\ell$ and timestamp $t_\ell$ into chunk signatures and verifies them against the auditor's metadata commitment. This ensures that only proofs generated with up-to-date metadata are accepted, thereby preventing *replay attacks* as defined in Section III-A.